
Comms-Library

Software Requirements Specification

VERSION	1.0.0
DATE	February 22, 2026
AUTHOR	Group - 16
STATUS	Draft

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Document Conventions	4
1.3	Intended Audience and Reading Suggestions	4
1.4	Project Scope	5
2	Overall Description	6
2.1	Product Perspective	6
2.2	Product Features	6
2.3	User Classes and Characteristics	6
2.3.1	Authors	6
2.3.2	Reviewers	7
2.3.3	Audience (End Users)	7
2.3.4	Administrators	7
3	System Architecture and Design Rationale	7
3.1	Architectural Overview	7
3.2	Design Rationale	8
3.3	Key Challenges Addressed and Solved	8
3.4	Architecture Diagrams	9
3.4.1	Edge Layer	9
3.4.2	Traffic Layer	9
3.4.3	Platform Layer	9
3.4.4	Gateway Layer	10
3.4.5	Application Layer	10
3.4.6	Service Layer	11
3.4.7	Intelligence Layer	11
3.4.8	Repository Layer	12
3.4.9	Integration Layer	12
3.4.10	Data Layer	13
4	Functional Requirements	14
4.1	Feature Priority Summary	14
4.2	Authors	16
4.2.1	F-AUT-001 State Management	16
4.2.2	F-AUT-002 Visibility Control	16
4.2.3	F-AUT-003 Tags	17
4.2.4	F-AUT-004 Co-authoring	18
4.2.5	F-AUT-005 Analysis	18
4.2.6	F-AUT-006 AI – Create Drafts	18
4.2.7	F-AUT-007 Reusable Component Creation	19
4.2.8	F-AUT-008 Citations & References	19
4.2.9	F-AUT-009 AI – Update Old Templates / Drafts	19
4.2.10	F-AUT-010 AI – Content Pipeline	20
4.2.11	F-AUT-011 History	20

4.2.12	F-AUT-012 AI – Auto Tag	21
4.3	Reviewers	21
4.3.1	F-REV-001 Approve / Deny	21
4.3.2	F-REV-002 Add Reviewers	22
4.3.3	F-REV-003 Comment	22
4.3.4	F-REV-004 Rollback Approvals	22
4.3.5	F-REV-005 AI – Automate Approval of Specific Format	23
4.3.6	F-REV-006 Assignment of Rules	23
4.3.7	F-REV-007 Comparison Between Versions	24
4.3.8	F-REV-008 Plagiarism	24
4.3.9	F-REV-009 AI – Screening	24
4.3.10	F-REV-010 AI – Content Contradiction	25
4.3.11	F-REV-011 AI – Comment to Action List	25
4.4	Audience	25
4.4.1	F-AUD-001 View	25
4.4.2	F-AUD-002 Copy	25
4.4.3	F-AUD-003 React / Like / Share / Comment	26
4.4.4	F-AUD-004 Bookmark	26
4.4.5	F-AUD-005 Progress	26
4.4.6	F-AUD-006 AI – Summarize	27
4.4.7	F-AUD-007 AI – Clustering	27
4.4.8	F-AUD-008 AI – RAG	27
4.4.9	F-AUD-009 Note Taking	28
4.4.10	F-AUD-010 Content Filtering	28
4.4.11	F-AUD-011 AI – AI Tutor	28
4.4.12	F-AUD-012 Info Graph	29
4.4.13	F-AUD-013 Related Content	29
4.4.14	F-AUD-014 AI – Convert Content Template	29
4.5	Collaboration (Troops)	29
4.5.1	F-COL-001 Peer Review	29
4.5.2	F-COL-002 Shared Space	29
4.5.3	F-COL-003 Q&A	29
4.6	Admin	30
4.6.1	F-ADM-001 Manage Roles	30
4.6.2	F-ADM-002 Manage User Groups	30
4.6.3	F-ADM-003 Monitor	30
4.6.4	F-ADM-004 Review Policy	31
4.6.5	F-ADM-005 Logs	31
4.6.6	F-ADM-006 AI – Monitor	31
4.6.7	F-ADM-007 Setup Pipelines	32
4.6.8	F-ADM-008 Invalidation	32
4.6.9	F-ADM-009 AI – Analyse	32
4.7	Template	33
4.7.1	F-TMP-001 CRUD	33
4.7.2	F-TMP-002 Clone	33
4.7.3	F-TMP-003 Add Channel	33

4.7.4	F-TMP-004 Translate	34
4.7.5	F-TMP-005 Editor	34
4.7.6	F-TMP-006 Formatting	34
4.7.7	F-TMP-007 AI – Draft First Version	35
4.7.8	F-TMP-008 Library (Reuse Components)	35
4.7.9	F-TMP-009 Hyperlink & Asset Management	35
4.7.10	F-TMP-010 Tag & Cluster	36
4.7.11	F-TMP-011 AI – Convert Between Channel Templates	36
4.7.12	F-TMP-012 Conditional Render	37
4.8	Channels	37
4.8.1	F-CHN-001 Email	37
4.8.2	F-CHN-002 SMS	37
4.8.3	F-CHN-003 Moodle	37
4.8.4	F-CHN-004 Canvas	37
4.8.5	F-CHN-005 Calendar	37
4.8.6	F-CHN-006 Push Notifications	37
4.8.7	F-CHN-007 WhatsApp	37
4.8.8	F-CHN-008 Social Media	37
4.8.9	F-CHN-009 RSS / Atom Feed	37
4.8.10	F-CHN-010 Website	37
4.9	Search	37
4.9.1	F-SRC-001 Content Check	37
4.9.2	F-SRC-002 Filtering	37
4.9.3	F-SRC-003 Full Text Search	38
4.9.4	F-SRC-004 AI – Smart Ranking	38
5	Non-Functional Requirements	39
5.1	Performance	39
5.2	Security	39
5.3	Availability	39
6	Sprint Wise Deliverables	40
7	Team	42
Appendix A	Glossary	43

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the Comms-Library. It details the functional and non-functional requirements for a platform designed to facilitate content creation, review, distribution, and consumption across multiple channels and user roles.

1.2 Document Conventions

The following conventions are used throughout this document:

- **Feature identifiers** : Functional features use the format `F-[SECTION]-[###]`, where `SECTION` is a three-letter code identifying the module and `###` is a zero-padded sequential number assigned independently of priority. Non-functional requirements use the format `NFR-CAT-NN`.
- **Priority levels** : Features are classified into three tiers: `M` Must Have, `S` Should Have, and `N` Nice to Have. In the feature tables (Section 3), features are grouped by priority column.
- **Callout boxes** : Blue boxes (`note`) provide supplementary context, green boxes (`tip`) highlight best-practice guidance, and amber boxes (`warning`) flag constraints or risks.
- **Typography** : `Monospaced text` denotes code identifiers, API endpoints, and configuration keys. **Bold** text is used for key terms on first use.

The following section prefixes are used for feature identifiers:

Prefix	Section	Scope
F-AUT	Authors	Content creation, drafting, tagging, co-authoring, and AI writing aids
F-REV	Reviewers	Approval workflows, comments, rollback, plagiarism, and AI screening
F-AUD	Audience	Viewing, bookmarking, reactions, progress tracking, and AI generators
F-ADM	Admin	Role management, pipelines, monitoring, logs, and AI analytics
F-CHN	Channels	Distribution channels - email, SMS, LMS, push, social media
F-TMP	Template	CRUD, editor, formatting, cloning, translation, and AI template tools
F-SRC	Search	Content search, filtering, full-text, and AI-powered retrieval
F-COL	Collaboration	Peer review, shared spaces, and Q&A

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the following audiences:

Reader	Recommended Sections
Project managers	Sections 1 and 2 for scope and product overview
Software developers	Sections 3 and 4 for architecture rationale and functional requirements
QA / Testers	Sections 4 and 5 for testable functional and non-functional requirements
Technical writers	Section 1 for document conventions, Section 2 for product context, and Appendix A for the Glossary
System architects	Section 3 for layered architecture, design rationale, and key engineering challenges
End users / Integrators	Sections 1.4 and 2.16 for high-level capabilities and product perspective
Administrators	Section 4.5 for Admin module requirements

All readers should begin with this Introduction and the Overall Description to establish context. Developers and testers should then proceed to Sections 3–5 for detailed requirements.

1.4 Project Scope

The Comms-Library aims to provide a comprehensive platform for educational institutions and organizations to create, manage, review, and distribute content across multiple channels. The system supports various user roles including Authors, Reviewers, Administrators, and end-user Audiences, with AI-powered features to enhance productivity and content quality.

2 Overall Description

2.1 Product Perspective

The Comms-Library is a new, self-contained platform designed to address the growing need for efficient content lifecycle management in educational and organizational contexts. The system integrates:

- Content authoring and collaborative editing capabilities
- Multi-stage review and approval workflows
- Multi-channel distribution (Email, SMS, LMS integration, social media, etc.)
- AI-powered content enhancement and automation
- Comprehensive administrative and monitoring tools

2.2 Product Features

The system provides eight major feature categories:

1. **Authors Module:** Comprehensive content creation tools including state management, tagging, co-authoring, content analysis, and AI-powered draft generation
2. **Reviewers Module:** Workflow management with approval/denial capabilities, multi-reviewer support, commenting, version comparison, and AI-assisted screening
3. **Audience Module:** Content consumption features including viewing, bookmarking, progress tracking, note-taking, and AI-powered summarization and tutoring
4. **Admin Module:** System administration including role management, user groups, monitoring, policy review, logging, and AI-powered analytics
5. **Channels Module:** Multi-channel distribution supporting email, SMS, LMS platforms (Moodle, Canvas), calendar integration, push notifications, WhatsApp, social media, RSS feeds, and websites
6. **Template Module:** Template management with CRUD operations, cloning, channel configuration, translation, rich text editing, formatting, and AI-assisted drafting
7. **Search Module:** Advanced search capabilities with content checking, filtering, full-text search, and AI-powered smart ranking
8. **Collaboration Module:** Peer review, shared workspaces, and Q&A functionality for team collaboration

2.3 User Classes and Characteristics

2.3.1 Authors

- **Primary Functions:** Content creation, editing, collaboration, template management
- **Privilege Level:** Can create and edit content, submit for review

2.3.2 Reviewers

- **Primary Functions:** Content review, approval, quality assurance, feedback provision
- **Privilege Level:** Can approve/deny content, add comments, manage review workflows

2.3.3 Audience (End Users)

- **Primary Functions:** Content consumption, learning, interaction
- **Privilege Level:** Read-only with interaction capabilities (comments, likes, bookmarks)

2.3.4 Administrators

- **Primary Functions:** System configuration, user management, monitoring, policy enforcement
- **Privilege Level:** Full system access

3 System Architecture and Design Rationale

3.1 Architectural Overview

The system is built as eleven clearly separated layers. Every inbound request passes through them in order from top to bottom.

- **Client Layer :** Web browsers, mobile apps, and API clients that initiate requests over HTTPS.
- **Edge Protection Layer :** The outermost defence - blocks DDoS attacks, filters bots, and enforces per-IP rate limits before any traffic reaches internal infrastructure.
- **Traffic Layer :** Terminates external TLS connections, then distributes the decrypted traffic evenly across backend instances via a reverse proxy and load balancer.
- **Platform Services :** A shared secret vault that supplies encryption keys and credentials to any layer that needs them, so no service ever stores secrets locally.
- **Gateway Layer :** The internal security checkpoint - sanitises payloads (stripping malformed or dangerous content), verifies caller identity, applies rate limits, and enforces AI usage quotas before routing the request inward.
- **Application Layer :** Coordinates high-level workflows - content authoring, review pipelines, real-time collaboration, and asset management and then forwards them to the Service Layer.
- **Service Layer :** The core of the system. Validates inputs, checks per-object permissions, enforces business rules, runs domain logic, and commits changes with transactional event records.
- **Intelligence Layer :** Handles all AI and search work in isolation, fast inline AI requests, long-running async AI jobs, and full-text or semantic search queries.
- **Repository Layer :** The only gateway to persistent storage. All reads and writes from the Application and Service layers go through dedicated repository components.
- **Integration Layer :** Manages background messaging, retries, notifications, third-party integrations, and dead-letter handling for failed events.

- **Data Layer :** The physical storage tier - cache, primary database, vector store, search index, audit log, and analytics sink each chosen for its specific access pattern.

3.2 Design Rationale

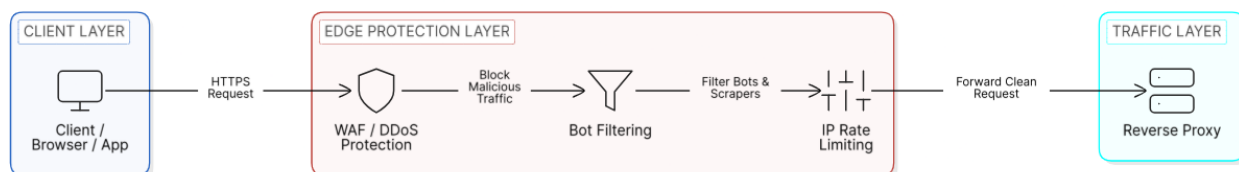
- **Security-in-Depth:** Traffic passes through three successive security checkpoints : the Edge Protection Layer, the Traffic Layer, and the Gateway Layer before any backend logic executes. This ensures that volumetric attacks, bot traffic, unauthenticated requests, and oversized AI usage are all rejected as early as possible, without burdening downstream services.
- **Centralised Secret Management:** Rather than embedding credentials in individual services, all secrets are fetched at runtime from a single Platform Services vault (P1). This eliminates secret sprawl and enables immediate rotation without redeployment.
- **Independent Scalability:** Separating the Application Layer (workflow orchestration) from the Service Layer (domain logic) allows either tier to be scaled independently. Adding new workflow types or channels in the Application Layer requires no changes to the core business rules in the Service Layer.
- **Workload Isolation:** AI workloads run in a dedicated Intelligence Layer. Synchronous fast-path requests (I1) are handled inline, while long-running jobs are offloaded to an asynchronous worker (I2). This prevents heavy AI computation from introducing latency into regular API calls.
- **Transactional Reliability:** The Outbox Writer (S5) and Outbox Repository (R5) guarantee that domain events are never silently lost: the event is committed atomically with the data change and is only forwarded to the Event Bus once the transaction succeeds.

3.3 Key Challenges Addressed and Solved

- **AI Resource Exhaustion and Latency:**
 - *Solution:* The Gateway Layer enforces AI usage limits, rejecting requests that exceed the quota early. Within the Intelligence Layer, quick tasks go to the *Sync AI (Fast Path)* and heavy long-running jobs go to the *Async AI Worker*, so neither blocks the other.
- **Keeping Data in Sync (Dual-Write Problem):**
 - *Solution:* Uses the **Transactional Outbox Pattern**. The Service Layer saves the data change and a notification event together in one step. The Integration Layer then sends the event to the *Event Bus* reliably, with retries if needed.
- **Fine-Grained Access Control:**
 - *Solution:* Uses a two-step permission check. The Gateway verifies who the user is, and the Service Layer checks whether they are allowed to access that specific piece of content.
- **Different Types of Data Queries:**
 - *Solution:* Uses different storage tools for different needs. Frequently accessed data goes to a *Cache (Redis)*, text searches use a *Search Index*, and AI-related lookups use a *Vector Store* for efficient question-answering (RAG).

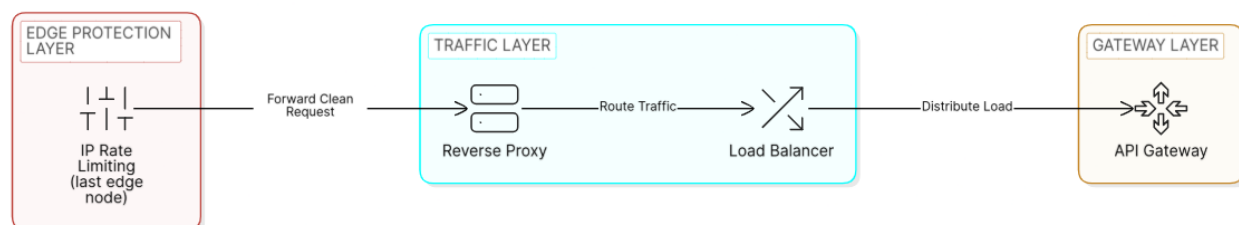
3.4 Architecture Diagrams

3.4.1 Edge Layer



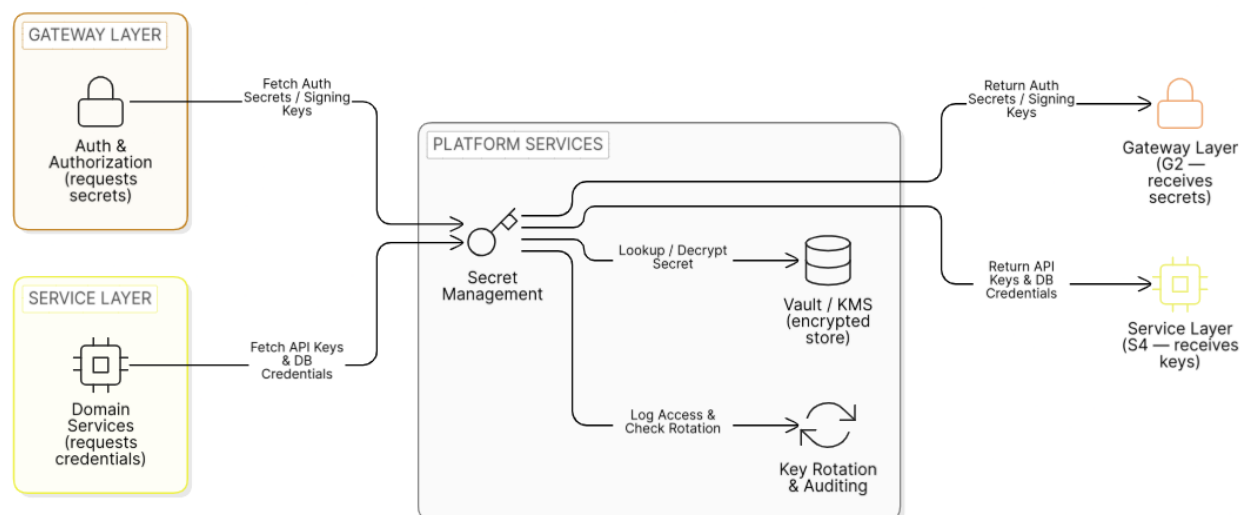
eraser

3.4.2 Traffic Layer



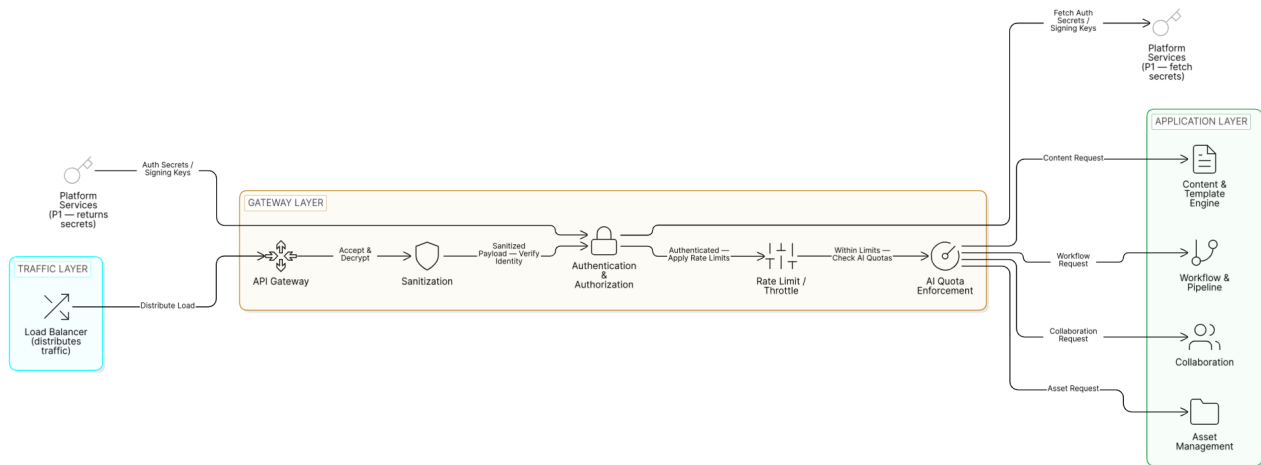
eraser

3.4.3 Platform Layer

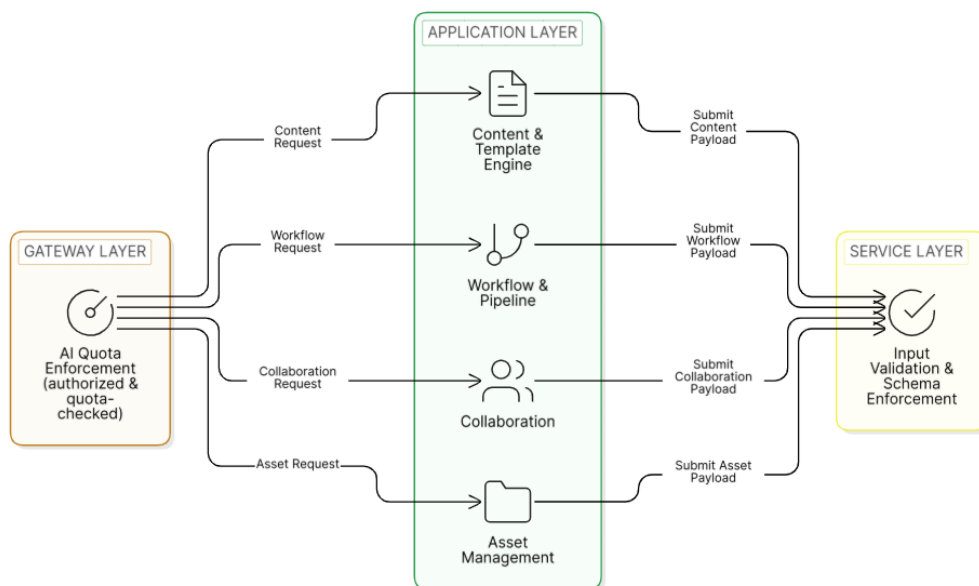


eraser

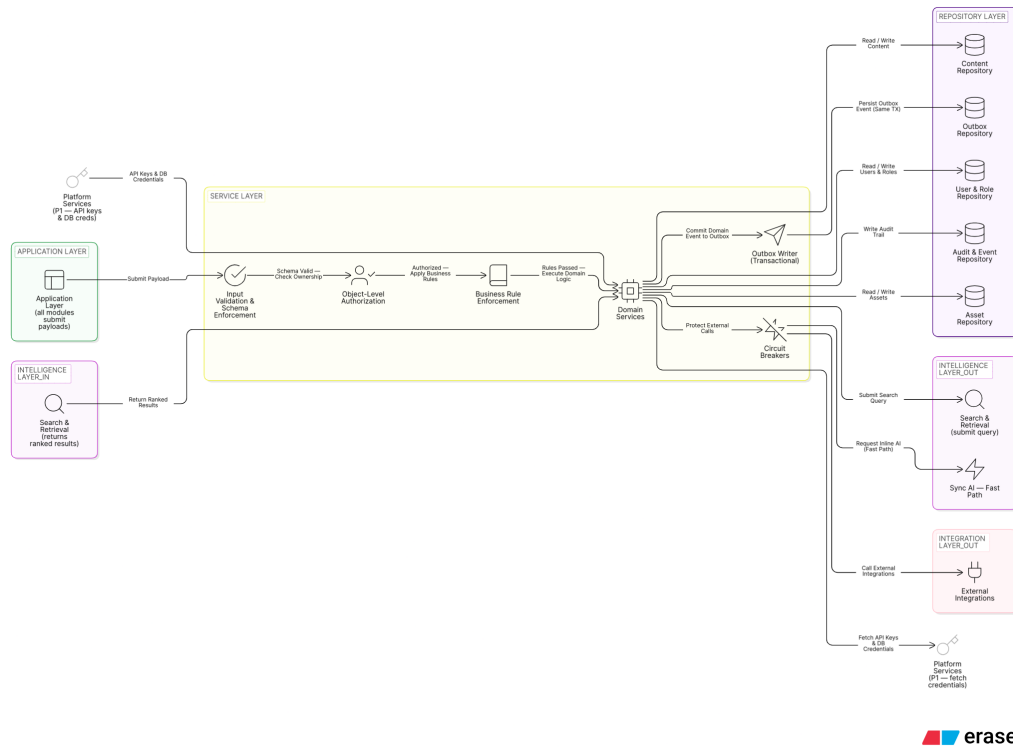
3.4.4 Gateway Layer



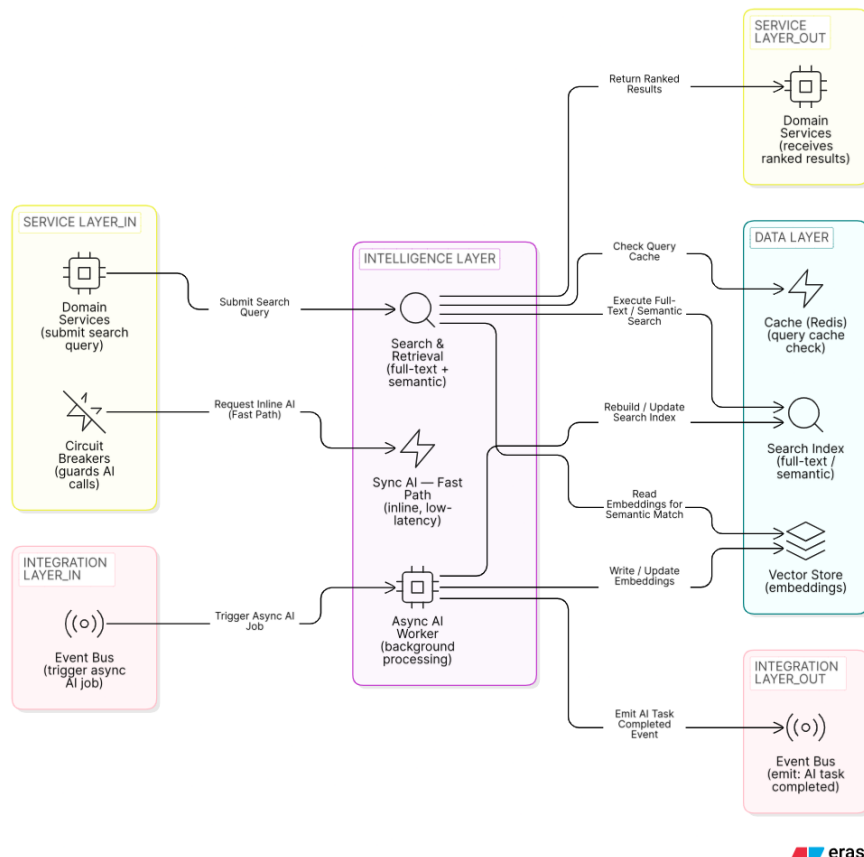
3.4.5 Application Layer



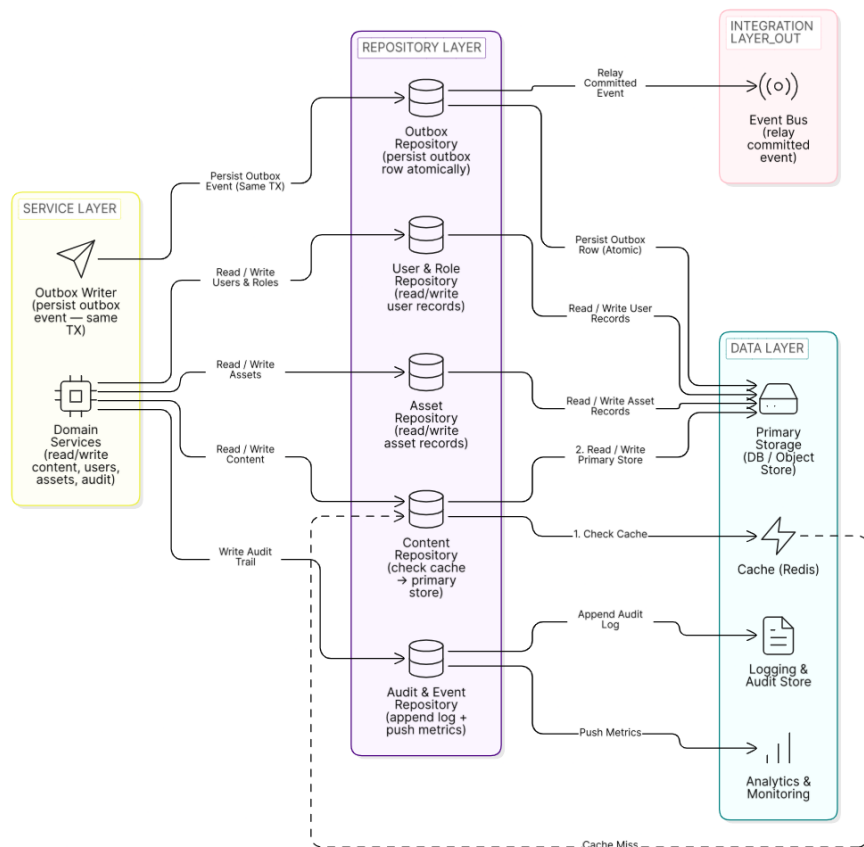
3.4.6 Service Layer



3.4.7 Intelligence Layer

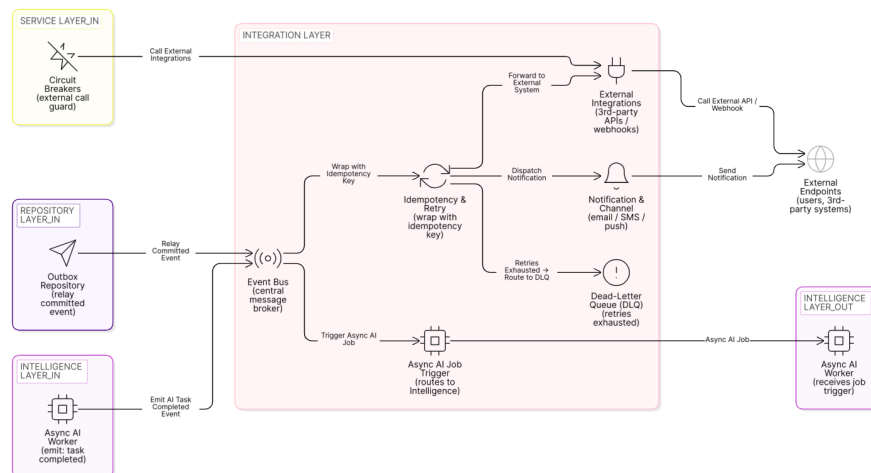


3.4.8 Repository Layer



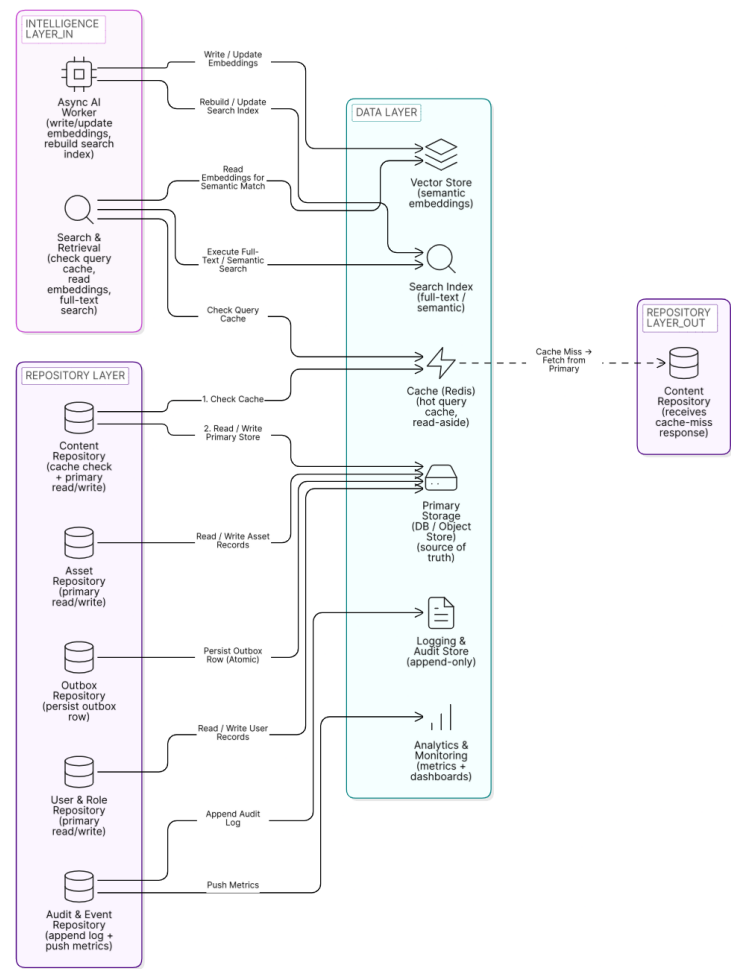
eraser

3.4.9 Integration Layer



eraser

3.4.10 Data Layer



4 Functional Requirements

4.1 Feature Priority Summary

The following table consolidates all functional features by module and priority tier.

Module	Must Have	Should Have	Nice to Have
Authors	• State Management • Visibility Control • Tags • Co-authoring • Analysis • AI – Create Drafts	• Reusable Component Creation • Citations & References • AI – Update Old Templates / Drafts • AI – Content Pipeline • History	• AI – Auto Tag
Reviewers	• Approve / Deny • Add Reviewers • Comment • Rollback Approvals • AI – Automate Approval of Specific Format	• Assignment of Rules • Comparison Between Versions • Plagiarism • AI – Screening	• AI – Content Contradiction • AI – Comment to Action List
Audience	• View • Copy • React / Like / Share / Comment • Bookmark • Progress • AI – Summarize • AI – Clustering • AI – RAG	• Note Taking • Content Filtering • AI – AI Tutor	• Info Graph • Related Content • AI – Convert Content Template
Admin	• Manage Roles • Manage User Groups • Monitor • Review Policy • Logs • AI – Monitor	• Setup Pipelines • Invalidation • AI – Analyse	—
Channels	• Email • SMS • Moodle • Canvas • Calendar • Push Notifications • WhatsApp • Social Media	• RSS / Atom Feed • Website	—

continued on next page

continued from previous page

Module	Must Have	Should Have	Nice to Have
Template	• CRUD • Clone • Add Channel • Translate • Editor • Formatting • AI – Draft First Version	• Library (Reuse Components) • Hyperlink & Asset Management • Tag & Cluster • AI – Convert Between Channel Templates	• Conditional Render
Search	• Content Check • Filtering • Full Text Search • AI – Smart Ranking	—	—
Collab.	—	—	• Peer Review • Shared Space • Q&A
Total	44 features	21 features	10 features

Feature priorities follow the convention defined in Section 1.2. IDs are assigned sequentially per section, independent of priority. Features are colour-coded: **Must Have**, **Should Have**, **Nice to Have**.

Standard Ingress Pipeline. Unless stated otherwise, every client-originated request traverses the following layers before reaching the Application Layer: **(1)** Edge Protection (WAF/DDoS Protection → Bot Filtering → IP Rate Limiting) → **(2)** Traffic (Reverse Proxy → Load Balancer) → **(3)** Gateway (API Gateway → TLS Termination & Payload Sanitisation → Authentication & Authorisation — retrieves signing keys from Secret Management — → Rate Limit / Throttle). AI-routed requests additionally pass through **AI Quota Enforcement** in the Gateway Layer. All stimulus/response sequences below begin after this pipeline has been traversed.

4.2 Authors

4.2.1 F-AUT-001 State Management

M

Description and Priority. Authors can create content and move it through its lifecycle states: Draft, In Review, Published, and Archived.

Stimulus/Response Sequences.

1. Author clicks “New Content” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Content & Template Engine forwards the request to Input Validation & Schema Enforcement → Object-Level Authorisation confirms author privileges → Business Rule Enforcement validates the initial state → Domain Services assigns a unique content ID → Content Repository persists the draft to Primary Storage → editor opens.
2. Author selects “Submit for Review” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Workflow & Pipeline forwards the request to Input Validation & Schema Enforcement → Object-Level Authorisation verifies ownership → Business Rule Enforcement validates the state transition and review policy → Domain Services updates the content state → Content Repository updates the state to In Review → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service alerts assigned reviewers.
3. Author or Admin archives published content → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation verifies permission → Content Repository transitions the item to Archived → Search Index de-lists it from active results → Audit & Event Repository logs the state change.

Functional Requirements.

- **REQ-1:** The system shall support at least four lifecycle states: Draft, In Review, Published, Archived.
- **REQ-2:** State transitions shall be validated against a configurable state machine. Invalid transitions shall be rejected with a descriptive error.
- **REQ-3:** Each state change shall be recorded in an audit log with timestamp, user, and previous state.

4.2.2 F-AUT-002 Visibility Control

M

Description and Priority. Authors can control who can see and access their content, separately from where it is in the publication process.

Visibility vs. Lifecycle State. Visibility is independent of the lifecycle state. A published item can still be Private or limited to a specific group. For example, a *Published + Private to Group* item is approved but only visible to group members.

Visibility Modes.

- **Public** — Discoverable and readable by any authenticated audience member.
- **Private** — Accessible only to the author(s) and assigned reviewers. Hidden from all audience views.
- **Hidden** — Excluded from all search indices and browse listings. Accessible only via a direct link by users who already hold the URL.
- **Archived** — Read-only and removed from active listings. Retained for audit and historical reference.
- **Private to Group** — Visible and accessible exclusively to members of one or more designated

user groups.

Stimulus/Response Sequences.

1. Author selects a visibility mode → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation verifies ownership → Business Rule Enforcement validates the transition (e.g. Draft cannot be set to Public before publication) → Content Repository updates the visibility metadata → Search Index re-evaluates discoverability → Cache (Redis) invalidates stale audience views → Audit & Event Repository logs the change.
2. Author chooses *Private to Group* → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Application Layer opens a group picker → Domain Services retrieves the available groups from the User & Role Repository → Input Validation & Schema Enforcement validates the selection → Business Rule Enforcement confirms the binding is permitted → Content Repository persists the group binding → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service optionally alerts group members.
3. User requests content outside their permitted scope → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation denies access and returns 403 without exposing the item's existence.

Functional Requirements.

- **REQ-1:** The system shall support five visibility modes: Public, Private, Hidden, Archived, and Private to Group. Enforcement shall occur at the Object-Level Authorisation layer, not solely at the UI.
- **REQ-2:** Hidden content shall be excluded from all Search Index queries and browse APIs regardless of the requester's role. Access shall only be possible via a direct, known ID.
- **REQ-3:** Every visibility change shall be recorded in the Audit & Event Repository with actor, timestamp, and previous and new modes.

4.2.3 F-AUT-003 Tags

M

Description and Priority. Authors can add, remove, and manage tags on their content to help organise and find it.

Stimulus/Response Sequences.

1. Author types a tag name → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Content & Template Engine forwards the query to Domain Services → Domain Services queries Search & Retrieval → Search & Retrieval fetches matching tags from the Search Index → Content & Template Engine auto-suggests existing tags and allows creation of new ones.
2. Author removes a tag → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement confirms the operation → Object-Level Authorisation verifies content ownership → Business Rule Enforcement validates the tag removal → Domain Services updates the tag list → Content Repository updates the content metadata → Search Index refreshes the tag facets.

Functional Requirements.

- **REQ-1:** The system shall allow authors to add, edit, and remove tags on any content they own.
- **REQ-2:** Tags shall be searchable and support hierarchical grouping.

4.2.4 F-AUT-004 Co-authoring

M

Description and Priority. Multiple authors can edit the same content at the same time. The system handles any conflicts automatically.

Stimulus/Response Sequences.

1. Author A and Author B open the same document → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Collaboration module requests a shared editing session → Object-Level Authorisation verifies both users have write access → Domain Services initialises the session and registers participants → Event Bus broadcasts the session-ready signal to all connected clients.
2. Author A types in paragraph 2 while Author B edits paragraph 5 → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Collaboration module forwards each delta to Input Validation & Schema Enforcement → Domain Services merges changes in real time via CRDT/OT → Content Repository persists the merged state → Event Bus broadcasts deltas to all connected participants.

Functional Requirements.

- **REQ-1:** The system shall support concurrent editing by at least 10 authors per document.
- **REQ-2:** Conflict resolution shall use CRDT or Operational Transform to guarantee eventual consistency.
- **REQ-3:** Each author's cursor and selection shall be visible to other participants in real time.

4.2.5 F-AUT-005 Analysis

M

Description and Priority. Authors can view how their content is performing, including who is reading it and how they are engaging with it.

Stimulus/Response Sequences.

1. Author opens the analytics dashboard → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms content ownership → Domain Services requests engagement data → Analytics & Monitoring returns views, likes, shares, and completion rates.
2. Author selects a date range → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services re-queries Analytics & Monitoring with the filter → system updates charts.

Functional Requirements.

- **REQ-1:** The system shall track views, unique readers, average reading time, and reaction counts per content item.
- **REQ-2:** Analytics data shall be exportable in CSV format.

4.2.6 F-AUT-006 AI – Create Drafts

M

Description and Priority. Authors can give the AI a topic or outline, and it will generate an initial draft for them.

Stimulus/Response Sequences.

1. Author provides a topic and optional outline → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Content & Template Engine

forwards the request to Input Validation & Schema Enforcement → Object-Level Authorisation confirms author privileges → Domain Services routes the prompt through Circuit Breakers to Sync AI (short prompts) or Async AI Worker (long-form generation).

2. AI returns generated text → Domain Services wraps the result as a new AI-generated draft → Content Repository persists it → Content & Template Engine opens the editor.

Functional Requirements.

- **REQ-1:** The system shall accept free-text prompts and structured outlines as input.
- **REQ-2:** Generated content shall be editable and carry an AI-generated metadata flag.
- **REQ-3:** The system shall allow the author to regenerate or refine sections iteratively.

4.2.7 F-AUT-007 Reusable Component Creation

S

Description and Priority. Authors can save sections or blocks of content as reusable components that can be inserted into other content items.

Stimulus/Response Sequences.

1. Author selects a block and clicks “Save as Component” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement checks the payload → Asset Management stores it in the component library → Asset Repository persists the component.
2. Author inserts a component into a new document → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services retrieves the component from the Asset Repository → Content & Template Engine embeds a linked or copied instance into the draft.

Functional Requirements.

- **REQ-1:** The system shall provide a component library accessible from the editor.
- **REQ-2:** Components shall support both linked (live-sync) and detached (snapshot) insertion modes.

4.2.8 F-AUT-008 Citations & References

S

Description and Priority. Authors can insert citations and manage a reference list within content items.

Stimulus/Response Sequences.

1. Author clicks “Insert Citation” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Content & Template Engine opens a search dialog → Domain Services queries Search & Retrieval → Search & Retrieval fetches matching references from the Search Index.
2. Author selects a reference → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the citation → Domain Services inserts the inline citation and appends it to the bibliography → Content Repository persists the update.

Functional Requirements.

- **REQ-1:** The system shall support standard citation formats (APA, IEEE, MLA).

4.2.9 F-AUT-009 AI – Update Old Templates / Drafts

S

Description and Priority. The AI reviews old content and suggests updates to bring it in line with current language, data, or standards.

Stimulus/Response Sequences.

1. Author selects “AI Refresh” on a legacy document → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Workflow & Pipeline forwards the request to Input Validation & Schema Enforcement → Object-Level Authorisation verifies content ownership → Domain Services routes the job through Circuit Breakers to the Async AI Worker → Async AI Worker scans for outdated references and phrasing using the Vector Store for semantic comparison.
2. Async AI Worker returns a change-set → Domain Services structures it as tracked changes → Content & Template Engine presents the tracked-changes view → accepted changes are persisted to the Content Repository via Domain Services.

Functional Requirements.

- **REQ-1:** The system shall identify sections with outdated information using configurable heuristics.
- **REQ-2:** All AI-suggested edits shall be presented as tracked changes, not applied directly.

4.2.10 F-AUT-010 AI – Content Pipeline**S**

Description and Priority. AI automates multi-step content preparation workflows, including grammar checking, formatting, compliance scanning, and scheduling.

Stimulus/Response Sequences.

1. Author triggers “Run Pipeline” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Workflow & Pipeline forwards the request to Input Validation & Schema Enforcement → Business Rule Enforcement validates the pipeline configuration → Domain Services executes each pipeline step sequentially, routing through Circuit Breakers to Sync AI (lightweight checks) or Async AI Worker (deep analysis) as appropriate.
2. Pipeline completes → Domain Services aggregates results → system displays a summary report with pass/fail status per step → Audit & Event Repository logs the pipeline run.

Functional Requirements.

- **REQ-1:** Pipelines shall be configurable with ordered steps (e.g. grammar, tone, compliance).
- **REQ-2:** Each step shall produce a pass/fail result with actionable feedback.

4.2.11 F-AUT-011 History**S**

Description and Priority. The system keeps a full history of all edits. Authors can browse past versions, compare any two side by side, and restore an older version as the new working draft. A snapshot is saved every time the author saves manually or when the content moves to a new state (e.g. Draft → In Review).

Stimulus/Response Sequences.

1. Author opens the History panel → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms ownership or reviewer access → Domain Services requests the snapshot list → Content Repository retrieves the list → system displays a timeline with version number, actor, timestamp, and change type.
2. Author selects a snapshot → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services fetches the payload via Content Repository → Content & Template Engine renders a read-only preview.
3. Author selects two snapshots for comparison → request traverses the standard ingress pipeline (Edge

Protection → Traffic → Gateway) → Domain Services fetches both from the Content Repository → Domain Services computes a word-level diff → system renders a side-by-side view with additions in green and deletions in red.

4. Author clicks “Restore” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Business Rule Enforcement checks the content is not locked → Domain Services writes the restored snapshot as a new head entry via Content Repository → Audit & Event Repository logs the action → editor opens on the restored content.
5. Save or state-transition event fires → Content Repository appends an immutable snapshot tagged with the event type.

Functional Requirements.

- **REQ-1:** The system shall create an immutable snapshot on every explicit save and on every lifecycle state transition. Snapshots shall never be deleted or overwritten.
- **REQ-2:** Each snapshot shall record the version number, actor identity, timestamp, and triggering event type.
- **REQ-3:** The diff view shall highlight changes at word level in both unified and side-by-side layouts. Restoring a snapshot shall append a new version entry and never truncate prior history.
- **REQ-4:** A restore shall be blocked if an active co-author session is open. The author shall be informed and prompted to retry once the session ends.

4.2.12 F-AUT-012 AI – Auto Tag

N

AI analyses content and suggests relevant tags automatically, drawn from the existing taxonomy. Suggestions are presented for author approval before being applied.

4.3 Reviewers

4.3.1 F-REV-001 Approve / Deny

M

Description and Priority. Reviewers can approve or deny content submissions, controlling the publication gate.

Stimulus/Response Sequences.

1. Reviewer opens a pending item → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation verifies reviewer assignment → Domain Services retrieves the content via Content Repository → system displays it with an Approve/Deny toolbar.
2. Reviewer clicks “Approve” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Business Rule Enforcement validates quorum and policy → Domain Services transitions the content to Published → Content Repository updates the state → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service notifies the author → Audit & Event Repository logs the decision.
3. Reviewer clicks “Deny” with a reason → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Business Rule Enforcement validates the denial → Domain Services transitions the item back to Draft → Content Repository updates the state → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service notifies the author with the denial reason → Audit & Event Repository logs the decision.

Functional Requirements.

- **REQ-1:** Approval and denial actions shall require a mandatory comment.
- **REQ-2:** The system shall log every approval decision with timestamp, reviewer, and rationale.

4.3.2 F-REV-002 Add Reviewers

M

Description and Priority. Reviewers or admins can assign one or more reviewers to a content item.

Stimulus/Response Sequences.

1. Author searches for a reviewer by name or role → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services queries the User & Role Repository → system displays matching users.
2. Author assigns selected reviewers → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the request → Business Rule Enforcement validates assignment rules → Domain Services records the assignment via Content Repository → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service sends review-request notifications.

Functional Requirements.

- **REQ-1:** The system shall support assigning multiple reviewers with independent approval tracks.
- **REQ-2:** Configurable quorum rules shall determine whether all or a subset of approvals are needed.

4.3.3 F-REV-003 Comment

M

Description and Priority. Reviewers can leave inline and general comments on content items.

Stimulus/Response Sequences.

1. Reviewer selects text and clicks “Comment” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement checks the payload → Domain Services persists the inline comment thread via Content Repository.
2. Author replies → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the reply → Domain Services appends it via Content Repository → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service notifies the reviewer.
3. Reviewer marks the comment as resolved → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the action → Domain Services updates the thread status via Content Repository → system collapses the thread.

Functional Requirements.

- **REQ-1:** Comments shall support rich text and @-mentions.
- **REQ-2:** Each comment thread shall have a Resolved / Open status.

4.3.4 F-REV-004 Rollback Approvals

M

Description and Priority. Reviewers or admins can undo a previous approval and move the content back to its pre-approved state.

Stimulus/Response Sequences.

1. Reviewer selects a published item and clicks “Rollback” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation verifies rollback permission → system prompts for a reason.
2. Reviewer confirms → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Business Rule Enforcement validates the rollback → Domain Services reverts the state to In Review → Content Repository updates the record → Audit & Event Repository logs the rollback with justification → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service notifies the author.

Functional Requirements.

- **REQ-1:** Rollback shall require a mandatory justification.
- **REQ-2:** The rolled-back version shall remain accessible in the version history.

4.3.5 F-REV-005 AI – Automate Approval of Specific Format M

Description and Priority. AI validates content against predefined format rules and auto-approves items that pass all checks.

Stimulus/Response Sequences.

1. Author submits content tagged with a format template → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the submission → Business Rule Enforcement checks format-template eligibility → Domain Services routes the validation job through Circuit Breakers to Sync AI (fast path) format validator.
2. All checks pass → Business Rule Enforcement confirms auto-approval eligibility → Domain Services auto-approves the item → Content Repository updates the state → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service notifies stakeholders → Audit & Event Repository logs the auto-approval.
3. One or more checks fail → Domain Services routes the item to human reviewers with the failure report attached → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service alerts the assigned reviewers.

Functional Requirements.

- **REQ-1:** Admins shall configure which templates are eligible for auto-approval.
- **REQ-2:** Auto-approved items shall be clearly labelled as such in the audit log.

4.3.6 F-REV-006 Assignment of Rules S

Description and Priority. Admins can define review rules that automatically assign reviewers based on content type, tags, or channel.

Stimulus/Response Sequences.

1. Admin creates a rule (e.g. “tag:policy → Legal team”) → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the rule syntax → Business Rule Enforcement registers the rule → Content Repository persists it.

2. Author submits content matching the rule → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Business Rule Enforcement evaluates all active rules → Workflow & Pipeline auto-assigns the configured reviewers → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service sends assignment notifications.

Functional Requirements.

- **REQ-1:** Rules shall support conditions on content type, tags, channel, and author group.
- **REQ-2:** Multiple rules may match. The system shall merge reviewer lists without duplication.

4.3.7 F-REV-007 Comparison Between Versions S

Description and Priority. Reviewers can compare any two versions of a content item side by side with highlighted differences.

Stimulus/Response Sequences.

1. Reviewer selects two versions from the history → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Content Repository retrieves both snapshots from Primary Storage → Domain Services computes the diff → system renders a diff view.
2. Reviewer navigates between changes → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Application Layer scrolls to the next/previous diff block.

Functional Requirements.

- **REQ-1:** The diff view shall highlight additions, deletions, and modifications at the paragraph and word level.
- **REQ-2:** The system shall support unified and side-by-side diff layouts.

4.3.8 F-REV-008 Plagiarism S

Description and Priority. Reviewers can trigger a plagiarism check during review to verify content originality.

Stimulus/Response Sequences.

1. Reviewer clicks “Run Plagiarism Check” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the request → Object-Level Authorisation confirms reviewer access → Domain Services routes the job through Circuit Breakers to the Async AI Worker → Async AI Worker analyses the content for similarity against the Vector Store and external sources.
2. Analysis completes → Domain Services embeds the similarity report in the review panel → Audit & Event Repository records the check result.

Functional Requirements.

- **REQ-1:** The plagiarism report shall be attached to the review record for audit purposes.
- **REQ-2:** The system shall flag items exceeding a configurable similarity threshold.

4.3.9 F-REV-009 AI – Screening S

Description and Priority. AI pre-screens content for policy violations and incomplete sections before human review begins.

Stimulus/Response Sequences.

1. Content enters the review queue → Workflow & Pipeline forwards a pre-screening request to Input Validation & Schema Enforcement → Business Rule Enforcement confirms screening is required for this content type → Domain Services routes the job through Circuit Breakers to the Async AI Worker.
2. Screening identifies issues → Async AI Worker returns findings → Domain Services attaches warnings to the review view with affected passages highlighted → Audit & Event Repository logs the screening result.

Functional Requirements.

- **REQ-1:** Screening shall check for policy compliance, completeness, and formatting rules.
- **REQ-2:** Results shall be advisory. The final decision remains with the human reviewer.

4.3.10 F-REV-010 AI – Content Contradiction

N

AI detects factual contradictions within a document or across related documents in the library, presenting pairs of conflicting statements with confidence scores and source references.

4.3.11 F-REV-011 AI – Comment to Action List

N

AI converts reviewer comments into a structured, prioritised action list for the author. Each action item links back to the originating comment and can be marked as done to resolve it.

4.4 Audience

4.4.1 F-AUD-001 View

M

Description and Priority. Audience members can view published content in a clean, responsive reading interface.

Stimulus/Response Sequences.

1. User navigates to a content item → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms the content is published and accessible → Content Repository retrieves it (Cache hit via Redis where available) → Content & Template Engine renders the published version.

Functional Requirements.

- **REQ-1:** The system shall render content in a responsive layout suitable for desktop and mobile.
- **REQ-2:** Only published content shall be visible to audience members.

4.4.2 F-AUD-002 Copy

M

Description and Priority. Audience members can copy content text to their clipboard for personal use.

Stimulus/Response Sequences.

1. User selects text and copies → request traverses the standard ingress pipeline → Client Layer intercepts the copy event → Content & Template Engine appends an attribution footer (unless disabled by admin configuration in the Content Repository) → system places plain or rich text on the clipboard.

Functional Requirements.

- **REQ-1:** Copied text shall include a source attribution unless disabled by the admin.

- **REQ-2:** Admins shall be able to disable copy on a per-content or per-channel basis.

4.4.3 F-AUD-003 React / Like / Share / Comment

M

Description and Priority. Audience members can react to, like, share, and comment on published content.

Stimulus/Response Sequences.

1. User clicks a reaction emoji → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement checks the payload → Domain Services records the reaction via Content Repository → Cache (Redis) refreshes the real-time display.
2. User clicks “Share” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Application Layer generates a shareable link or share dialog → Domain Services routes the cross-platform posting request through Circuit Breakers to External Integrations.
3. User posts a comment → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement sanitises the input → Domain Services appends the comment via Content Repository → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service notifies subscribed participants.

Functional Requirements.

- **REQ-1:** The system shall support at least five reaction types and a Like toggle.
- **REQ-2:** Comments shall support threading up to two levels deep.

4.4.4 F-AUD-004 Bookmark

M

Description and Priority. Users can bookmark content items for quick access later.

Stimulus/Response Sequences.

1. User clicks the bookmark icon → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the request → Domain Services saves the bookmark via Content Repository.
2. User opens “My Bookmarks” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms the user’s identity → Domain Services retrieves the bookmarked items via Content Repository → system displays them sorted by date.

Functional Requirements.

- **REQ-1:** Bookmarks shall be organised into user-defined folders.
- **REQ-2:** The system shall notify the user when a bookmarked item is updated.

4.4.5 F-AUD-005 Progress

M

Description and Priority. The system tracks the audience member’s reading progress (Done, Reading, Not Started) across all content.

Stimulus/Response Sequences.

1. User scrolls through content → scroll position is periodically batched and sent → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the progress event → Domain Services marks sections as read via Content Repository.

2. User opens their dashboard → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms the user's identity → Domain Services queries Analytics & Monitoring → system displays a progress overview with completion percentages.

Functional Requirements.

- **REQ-1:** Progress shall be synced across devices for the same user account.
- **REQ-2:** The system shall support manual status overrides (mark as done / unread).

4.4.6 F-AUD-006 AI – Summarize

M

Description and Priority. AI generates a concise summary of a content item at adjustable detail levels.

Stimulus/Response Sequences.

1. User clicks “Summarize” and selects a length → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the request → Domain Services routes the job through Circuit Breakers to Sync AI (short content) or Async AI Worker (long-form).
2. AI returns the summary → Domain Services formats the result → Content & Template Engine displays it in a collapsible panel → Cache (Redis) stores it for subsequent requests.

Functional Requirements.

- **REQ-1:** Summaries shall preserve key facts, figures, and conclusions from the original.
- **REQ-2:** The user shall be able to copy or export the summary independently.

4.4.7 F-AUD-007 AI – Clustering

M

Description and Priority. AI groups content into clusters to help users discover material by topic.

Stimulus/Response Sequences.

1. User opens the “Explore” page → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Application Layer forwards the request to Domain Services → Domain Services submits a cluster query to Search & Retrieval → Search & Retrieval reads embeddings from the Vector Store → Async AI Worker computes topic clusters → system displays AI-generated topic clusters.
2. User selects a cluster → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Domain Services queries Search & Retrieval → Search & Retrieval retrieves all content within that cluster from the Search Index → system lists items ranked by relevance.

Functional Requirements.

- **REQ-1:** Clusters shall be regenerated periodically as new content is published.
- **REQ-2:** Users shall be able to follow a cluster to receive notifications on new additions.

4.4.8 F-AUD-008 AI – RAG

M

Description and Priority. Retrieval-Augmented Generation allows users to ask natural-language questions and receive answers grounded in library content.

Stimulus/Response Sequences.

1. User types a question in the AI assistant → request traverses the standard ingress pipeline (Edge

Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the query → Domain Services queries Search & Retrieval → Search & Retrieval fetches relevant passages from the Vector Store and Search Index → Domain Services routes through Circuit Breakers to Sync AI (fast path) → Sync AI generates a grounded answer.

2. System displays the answer with inline citations → Domain Services verifies citation validity via Content Repository.

Functional Requirements.

- **REQ-1:** Every AI-generated answer shall include citations to the source documents.
- **REQ-2:** The retrieval index shall be updated within 5 minutes of content publication.

4.4.9 F-AUD-009 Note Taking S

Description and Priority. Audience members can take private notes alongside published content.

Stimulus/Response Sequences.

1. User highlights text and clicks “Add Note” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement checks the note content → a note editor opens anchored to the selection.
2. User saves the note → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services persists it via Content Repository → system shows a margin indicator at the anchored position.

Functional Requirements.

- **REQ-1:** Notes shall be private to the user and not visible to authors or other audience members.
- **REQ-2:** Notes shall be exportable in Markdown and PDF formats.

4.4.10 F-AUD-010 Content Filtering S

Description and Priority. Users can filter and selectively export portions of content by section, tag, or keyword.

Stimulus/Response Sequences.

1. User applies filters (tags, sections, date range) → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services forwards the filter query to Search & Retrieval → Search & Retrieval applies faceted filtering via the Search Index → system displays matching content blocks.
2. User clicks “Export Selection” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Content & Template Engine generates a PDF/Markdown file → Asset Management stores the export temporarily.

Functional Requirements.

- **REQ-1:** Filters shall be combinable with AND/OR logic.
- **REQ-2:** Exported documents shall retain formatting and include source attribution.

4.4.11 F-AUD-011 AI – AI Tutor S

Description and Priority. An interactive AI tutor answers questions about the content in a conversational interface, grounded in library material.

Stimulus/Response Sequences.

1. User opens the AI Tutor panel and asks a question → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the query → Domain Services queries Search & Retrieval → Search & Retrieval fetches relevant context from the Vector Store → Domain Services routes through Circuit Breakers to Sync AI → Sync AI generates a grounded answer.
2. User asks a follow-up → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Domain Services retrieves conversational context from Cache (Redis) → Domain Services routes through Circuit Breakers to Sync AI → Sync AI refines its response with additional retrieval.

Functional Requirements.

- **REQ-1:** Responses shall be grounded exclusively in the library's published content (no hallucination).
- **REQ-2:** Conversation history shall be saved and resumable.

4.4.12 F-AUD-012 Info Graph N

A visual knowledge graph showing relationships between content items as an interactive, zoomable node-link diagram (Obsidian-style).

4.4.13 F-AUD-013 Related Content N

The system recommends 3–5 related content items at the end of each page, based on shared tags, audience overlap, and semantic similarity.

4.4.14 F-AUD-014 AI – Convert Content Template N

AI re-formats content from one template to another (e.g. lecture notes to slide deck), preserving textual content while adapting layout.

4.5 Collaboration (Troops)

4.5.1 F-COL-001 Peer Review N

Users can request and conduct peer reviews within collaborative workgroups, with inline annotations, overall feedback, and visible review status.

4.5.2 F-COL-002 Shared Space N

Workgroups have a shared content space for collaborative drafting and resource pooling, with configurable access levels and a unified activity feed.

4.5.3 F-COL-003 Q&A N

A question-and-answer forum within workgroups for discussing content and resolving queries, with threading, accepted answers, and upvote/downvote.

4.6 Admin

4.6.1 F-ADM-001 Manage Roles

M

Description and Priority. Admins can create, edit, and delete roles with fine-grained permission sets.

Stimulus/Response Sequences.

1. Admin opens Role Management → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms admin privileges → Domain Services retrieves all roles via User & Role Repository → system displays the role list with permissions.
2. Admin promotes or demotes a user → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the permission set → Object-Level Authorisation confirms admin privileges → Business Rule Enforcement ensures no privilege escalation → Domain Services persists the change via User & Role Repository.

Functional Requirements.

- **REQ-1:** The system shall ship with default roles (Admin, Author, Reviewer, Audience).
- **REQ-2:** Permissions shall be granular (per module and per action: create, read, update, delete).

4.6.2 F-ADM-002 Manage User Groups

M

Description and Priority. Admins can organise users into groups for bulk permission assignment and content targeting.

Stimulus/Response Sequences.

1. Admin creates a group and adds members → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the configuration → Object-Level Authorisation confirms admin privileges → Domain Services persists the group and assigns the role to all members via User & Role Repository.
2. Admin removes a user from a group → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the request → Domain Services revokes the group's permissions via User & Role Repository → Domain Services invalidates that user's session token in Cache (Redis).

Functional Requirements.

- **REQ-1:** Users may belong to multiple groups.
- **REQ-2:** Groups shall support bulk import from CSV or LDAP/SAML directory sync.

4.6.3 F-ADM-003 Monitor

M

Description and Priority. Admins have access to a real-time system dashboard showing content volume, user activity, and pipeline throughput.

Stimulus/Response Sequences.

1. Admin opens the Monitor dashboard → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms admin access → Analytics & Monitoring returns live KPIs → system displays real-time charts and metrics.

Functional Requirements.

- **REQ-1:** The dashboard shall display active users, content in each state, and pipeline bottlenecks.

4.6.4 F-ADM-004 Review Policy

M

Description and Priority. Admins can define and enforce review policies (e.g. mandatory review for certain content types).

Stimulus/Response Sequences.

1. Admin creates a policy rule → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the rule syntax → Object-Level Authorisation confirms admin privileges → Business Rule Enforcement registers the policy → Content Repository persists the rule.
2. Author tries to publish without meeting the policy → request traverses the standard ingress pipeline → Business Rule Enforcement evaluates the review policy → system blocks the action and displays the unmet requirement.

Functional Requirements.

- **REQ-1:** Policies shall be configurable per content type, channel, and user group.
- **REQ-2:** Policy violations shall produce clear, actionable error messages.

4.6.5 F-ADM-005 Logs

M

Description and Priority. The system maintains comprehensive audit logs of all administrative and content-lifecycle actions.

Stimulus/Response Sequences.

1. Admin opens the Audit Log viewer and applies filters → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms admin access → Audit & Event Repository queries the Logging & Audit Store → system displays matching log entries.

Functional Requirements.

- **REQ-1:** Logs shall be immutable and retained for a configurable period (default: 2 years).
- **REQ-2:** Logs shall be filterable by user, action type, content ID, and date range.
- **REQ-3:** Logs shall be exportable in CSV and JSON formats.

4.6.6 F-ADM-006 AI – Monitor

M

Description and Priority. AI-powered monitoring detects anomalies in content volume, user behaviour, or system performance and raises alerts.

Stimulus/Response Sequences.

1. Async AI Worker continuously analyses metrics from Analytics & Monitoring → Async AI Worker detects an anomaly (e.g. sudden spike in content creation) and emits an alert event to the Event Bus → Idempotency & Retry wraps the dispatch → Notification & Channel service sends alerts to the admin dashboard and via email → Audit & Event Repository logs the anomaly event.

Functional Requirements.

- **REQ-1:** Alerting thresholds shall be configurable per metric.
- **REQ-2:** Alerts shall include a summary of the anomaly and suggested investigation steps.

4.6.7 F-ADM-007 Setup Pipelines

S

Description and Priority. Admins can define content publication pipelines with ordered stages (e.g. Draft → Review → Legal → Publish).

Stimulus/Response Sequences.

1. Admin opens Pipeline Builder → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Workflow & Pipeline renders a drag-and-drop stage editor with available pipeline components.
2. Admin adds stages and assigns roles → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the configuration → Workflow & Pipeline registers it → Content Repository persists the pipeline definition.

Functional Requirements.

- **REQ-1:** Pipelines shall support sequential and parallel stages.
- **REQ-2:** Each stage shall have configurable timeout and escalation rules.

4.6.8 F-ADM-008 Invalidation

S

Description and Priority. Admins can invalidate (unpublish) content that is outdated, incorrect, or violates policy.

Stimulus/Response Sequences.

1. Admin selects content and clicks “Invalidate” with a reason → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms admin permission → Business Rule Enforcement validates the action → Content Repository marks the content as invalidated → Search Index removes it from public results → Outbox Writer stages a notification event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service notifies the author → Audit & Event Repository logs the invalidation.

Functional Requirements.

- **REQ-1:** Invalidated content shall be hidden from audience but retained in the archive.
- **REQ-2:** Bulk invalidation shall be supported via multi-select.

4.6.9 F-ADM-009 AI – Analyse

S

Description and Priority. AI provides analytical insights on content quality, readability scores, and audience engagement trends.

Stimulus/Response Sequences.

1. Admin opens AI Analytics → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Object-Level Authorisation confirms admin access → Domain Services routes the analysis job through Circuit Breakers to the Async AI Worker → Async AI Worker computes readability scores and engagement trends from Analytics & Monitoring data → system displays heatmaps and trend charts.

Functional Requirements.

- **REQ-1:** Analyses shall include readability index, average engagement time, and drop-off points.
- **REQ-2:** Reports shall be schedulable for periodic email delivery.

4.7 Template

4.7.1 F-TMP-001 CRUD

M

Description and Priority. Full Create, Read, Update, Delete operations on content templates.

Stimulus/Response Sequences.

1. Author clicks “New Template” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the creation request → Object-Level Authorisation confirms author privileges → Domain Services initialises a blank template record → Content & Template Engine opens the canvas editor.
2. Author saves changes → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the template structure → Object-Level Authorisation confirms ownership → Business Rule Enforcement enforces template integrity rules → Domain Services persists the template via Content Repository.

Functional Requirements.

- **REQ-1:** Templates shall define section structure, required fields, and default formatting.
- **REQ-2:** Deleting a template in use shall require migrating existing content to another template first.

4.7.2 F-TMP-002 Clone

M

Description and Priority. Users can clone an existing template to use as a starting point for a new one.

Stimulus/Response Sequences.

1. User selects a template and clicks “Clone” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the request → Object-Level Authorisation verifies read permission on the source template → Business Rule Enforcement confirms the template is eligible for cloning → Domain Services creates a deep copy in Draft state with a new ID → Content Repository persists the clone independently.

Functional Requirements.

- **REQ-1:** Cloned templates shall be independent. Changes shall not affect the original.

4.7.3 F-TMP-003 Add Channel

M

Description and Priority. Templates can be associated with one or more distribution channels, defining how content is rendered per channel.

Stimulus/Response Sequences.

1. Author opens a template and clicks “Add Channel” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services retrieves the list of available channels via the Notification & Channel service → system displays the channel list.
2. Author selects a channel and configures the channel-specific layout → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the binding → Object-Level Authorisation confirms template ownership → Business Rule Enforcement ensures the channel is supported by the template type → Domain Services saves the channel binding via Content Repository.

Functional Requirements.

- **REQ-1:** Each channel binding shall have its own rendering configuration (layout, fields, media handling).

4.7.4 F-TMP-004 Translate

M

Description and Priority. Template labels, placeholders, and static text can be translated into multiple languages.

Stimulus/Response Sequences.

1. Author selects “Manage Translations” → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms template ownership → Domain Services extracts all localisable keys and retrieves translations via Content Repository → Content & Template Engine displays the string table.
2. Author enters translations per language → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the entries → Object-Level Authorisation confirms ownership → Business Rule Enforcement validates completeness against required language set → Domain Services persists the translations via Content Repository for render-time use.

Functional Requirements.

- **REQ-1:** The system shall support i18n key-value pairs per template and per language.
- **REQ-2:** Missing translations shall fall back to the default language.

4.7.5 F-TMP-005 Editor

M

Description and Priority. A visual WYSIWYG editor for designing and modifying template layouts.

Stimulus/Response Sequences.

1. Author opens the template editor → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms template ownership → Domain Services loads the component palette from the Asset Repository → Content & Template Engine renders the drag-and-drop canvas with the component palette.
2. Author rearranges sections and saves → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the layout → Object-Level Authorisation confirms ownership → Business Rule Enforcement validates structural constraints → Domain Services persists the template definition via Content Repository.

Functional Requirements.

- **REQ-1:** The editor shall support drag-and-drop section reordering, rich-text blocks, and media placeholders.
- **REQ-2:** Templates shall be saveable as drafts and previewable before activation.

4.7.6 F-TMP-006 Formatting

M

Description and Priority. Templates define formatting rules (fonts, colours, spacing) that are enforced on all content using the template.

Stimulus/Response Sequences.

1. Author edits the template’s style settings → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the style values → Content & Template Engine applies them to the live preview (no persistence at this

stage).

2. Author creates content using the template → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the payload → Object-Level Authorisation confirms author access → Business Rule Enforcement enforces the template's formatting rules → Domain Services persists the styled content via Content Repository.

Functional Requirements.

- **REQ-1:** Formatting rules shall cover typography, colour palette, heading styles, and media constraints.
- **REQ-2:** Authors shall not be able to override template-locked styles.

4.7.7 F-TMP-007 AI – Draft First Version

M

Description and Priority. AI generates a first-draft template layout based on a description of the desired content structure.

Stimulus/Response Sequences.

1. Author describes the template requirements in natural language → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the prompt → Object-Level Authorisation confirms author privileges → Domain Services routes the generation request through Circuit Breakers to the Async AI Worker → Async AI Worker generates a template with sections, fields, and formatting → Domain Services persists the generated draft via Content Repository → Content & Template Engine opens it in the editor.

Functional Requirements.

- **REQ-1:** The generated template shall be fully editable in the standard template editor.
- **REQ-2:** AI shall explain its design choices in an attached rationale note.

4.7.8 F-TMP-008 Library (Reuse Components)

S

Description and Priority. Authors can insert pre-built, reusable components (e.g. standard headers, legal footers, or shared info-blocks) directly into templates. Components support both live-linked and snapshot modes, ensuring global updates propagate automatically when required.

Stimulus/Response Sequences.

1. Author drags a library component into the template editor → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Domain Services retrieves the component from the Asset Repository → Content & Template Engine embeds a linked instance.
2. Library component is updated → Async AI Worker emits a component-update event to the Event Bus → Idempotency & Retry wraps the dispatch → Domain Services receives the propagation event and updates all linked templates via Content Repository.

Functional Requirements.

- **REQ-1:** Components shall support linked (auto-update) and detached (snapshot) modes.
- **REQ-2:** The library shall be searchable and categorised by component type.

4.7.9 F-TMP-009 Hyperlink & Asset Management

S

Description and Priority. Templates can manage internal/external hyperlinks and embedded assets (images, documents) with integrity checking.

Stimulus/Response Sequences.

1. Author opens the asset manager → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Object-Level Authorisation confirms template ownership → Domain Services retrieves all linked assets from the Asset Repository → Domain Services checks link status (valid, broken, pending) via External Integrations for external URLs → system displays the asset list.
2. Domain Services detects a broken link during a periodic scan → Outbox Writer stages an alert event → Event Bus relays the committed event → Idempotency & Retry wraps the dispatch → Notification & Channel service alerts the admin or author → admin/author can replace or remove the broken asset.

Functional Requirements.

- **REQ-1:** The system shall periodically scan for broken internal and external links.
- **REQ-2:** Assets shall be stored in a managed repository with versioning.

4.7.10 F-TMP-010 Tag & Cluster**S**

Description and Priority. Templates can be tagged and grouped into clusters for easier discovery and management.

Stimulus/Response Sequences.

1. Author assigns tags to a template → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the tags → Object-Level Authorisation confirms template ownership → Business Rule Enforcement validates tags against the shared taxonomy → Domain Services updates the template metadata via Content Repository → Search Index refreshes the tag facets for filtered searches.

Functional Requirements.

- **REQ-1:** Tags shall be shared across the content and template taxonomies.

4.7.11 F-TMP-011 AI – Convert Between Channel Templates**S**

Description and Priority. AI automatically adapts a template's layout from one channel format to another (e.g. email to SMS).

Stimulus/Response Sequences.

1. Author selects a source template and a target channel → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the selection → Object-Level Authorisation confirms template access → Business Rule Enforcement verifies the target channel is supported → Domain Services routes the conversion job through Circuit Breakers to the Async AI Worker → Async AI Worker generates a converted template draft adapted to the target channel's constraints → Domain Services persists the draft via Content Repository → Content & Template Engine opens it for editing.

Functional Requirements.

- **REQ-1:** Conversion shall preserve content structure and adapt media/layout to the target channel's constraints.
- **REQ-2:** The converted template shall be editable before activation.

4.7.12 F-TMP-012 Conditional Render

N

Conditional rendering of template sections based on audience group, grade, or role. Hidden sections are completely absent from the rendered output.

4.8 Channels

4.8.1 F-CHN-001 Email

M

4.8.2 F-CHN-002 SMS

M

4.8.3 F-CHN-003 Moodle

M

4.8.4 F-CHN-004 Canvas

M

4.8.5 F-CHN-005 Calendar

M

4.8.6 F-CHN-006 Push Notifications

M

4.8.7 F-CHN-007 WhatsApp

M

4.8.8 F-CHN-008 Social Media

M

4.8.9 F-CHN-009 RSS / Atom Feed

S

4.8.10 F-CHN-010 Website

S

4.9 Search

4.9.1 F-SRC-001 Content Check

M

Description and Priority. Users can verify whether a specific piece of content exists in the library.

Stimulus/Response Sequences.

1. User enters a title or excerpt → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the query → Domain Services forwards the request to Search & Retrieval → Search & Retrieval queries the Search Index across titles, summaries, and body text → system returns matching results or “No results found.”

Functional Requirements.

- **REQ-1:** The check shall search across titles, summaries, and body text.
- **REQ-2:** Results shall be returned within 500 ms for typical queries.

4.9.2 F-SRC-002 Filtering

M

Description and Priority. Users can apply faceted filters (tags, author, date, channel, status) to narrow search results.

Stimulus/Response Sequences.

1. User applies one or more filters → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the filter parameters → Domain Services forwards the request to Search & Retrieval → Search & Retrieval checks Cache (Redis) for repeated queries then applies faceted filtering via the Search Index → system updates results in real time.

Functional Requirements.

- **REQ-1:** Filters shall be combinable. The system shall display the active filter count.

- **REQ-2:** Filter state shall be reflected in the URL for shareability.

4.9.3 F-SRC-003 Full Text Search

M

Description and Priority. The system provides full-text search across all published content with relevance ranking.

Stimulus/Response Sequences.

1. User types a query → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway) → Input Validation & Schema Enforcement validates the query string → Domain Services forwards the request to Search & Retrieval → Search & Retrieval performs a full-text search on the Search Index with stemming and typo tolerance → results are ranked by relevance and returned with highlighted matches.

Functional Requirements.

- **REQ-1:** Search shall support stemming, synonym expansion, and typo tolerance.
- **REQ-2:** The index shall be updated within 5 minutes of content publication or modification.

4.9.4 F-SRC-004 AI – Smart Ranking

M

Description and Priority. AI ranks search results using semantic understanding, link analysis, and engagement signals to surface the most relevant content.

Stimulus/Response Sequences.

1. User enters a conceptual query → request traverses the standard ingress pipeline (Edge Protection → Traffic → Gateway with AI Quota Enforcement) → Input Validation & Schema Enforcement validates the query → Domain Services forwards the request to Search & Retrieval → Search & Retrieval fetches candidates from the Search Index and reads embeddings from the Vector Store → Domain Services routes the ranking request through Circuit Breakers to Sync AI → Sync AI computes semantic similarity scores → results are blended with keyword relevance, recency, and engagement signals → system returns ranked results.

Functional Requirements.

- **REQ-1:** Ranking shall combine vector-based semantic similarity with link density, view count, recency, and reaction count.
- **REQ-2:** The system shall maintain a vector index of all published content, re-computed at least daily.
- **REQ-3:** Semantic and keyword result weights shall be configurable by the admin.

5 Non-Functional Requirements

5.1 Performance

ID	Requirement	Priority
NFR-PERF-01	End-to-end request latency shall not exceed 100 ms at p95 and 300 ms at p99 under nominal load	HIGH
NFR-PERF-02	The system shall support concurrent read and write operations without lock contention. Read throughput shall not degrade by more than 10% under simultaneous write load	HIGH
NFR-PERF-03	The system shall support 1000 concurrent writes	HIGH

5.2 Security

- NFR-SEC-01

HIGH

All data in transit shall be encrypted via TLS 1.3
- NFR-SEC-02

HIGH

Symmetric encryption shall use AES-256, with keys derived via HKDF and managed through a dedicated secrets manager (e.g. AWS KMS). Keys must be rotated at regular intervals.
- NFR-SEC-03

MED

Authentication tokens shall never be written to persistent logs. Log pipelines shall apply token-scrubbing filters before any data reaches the audit or analytics store.
- NFR-SEC-04

MED

The library shall enforce rate limiting on connection attempts to mitigate abuse. Limits shall be applied before authentication to avoid burning compute on flood traffic.

5.3 Availability

Uptime target: 99.9% availability per calendar month ($\approx$ 8.7 hours downtime/year). The library shall implement automatic reconnection to mask transient server failures from the host application.

To sustain availability at a 3-nines scale, the following infrastructure guarantees shall be in place:

- NFR-AVL-01

HIGH

Load balancers shall distribute traffic across all active nodes with health-check intervals of small duration, automatically rerouting requests away from degraded instances without manual intervention.
- NFR-AVL-02

HIGH

All primary data stores shall maintain synchronous replication to at least one standby replica within the same region, with asynchronous cross-region replication for disaster recovery, failover shall complete within 30 seconds.

6 Sprint Wise Deliverables

The following table summarises the sprint-wise scope across four sprints. All **Must Have** and **Should Have** features are committed deliverables. Only the 10 **Nice to Have** features appear as stretch goals, spread evenly across sprints and delivered if capacity allows. Features are sequenced so that dependencies are satisfied before the features that rely on them.

Sprint	Focus	Deliverables
Sprint II	<i>Foundation: Auth, Admin & Core Authoring</i>	Committed (14 features): • F-AUT-001 State Management • F-AUT-002 Visibility Control • F-AUT-003 Tags • F-AUT-004 Co-authoring • F-AUT-006 AI – Create Drafts • F-ADM-001 Manage Roles • F-ADM-002 Manage User Groups • F-ADM-004 Review Policy • F-ADM-005 Logs • F-REV-001 Approve / Deny • F-REV-002 Add Reviewers • F-REV-003 Comment • F-REV-004 Rollback Approvals • F-REV-005 AI – Automate Approval of Specific Format Stretch — Nice to Have (2): • F-AUT-012 AI – Auto Tag • F-REV-010 AI – Content Contradiction
Sprint III	<i>Templates, Search & Authoring+</i>	Committed (15 features): • F-TMP-001 CRUD • F-TMP-002 Clone • F-TMP-003 Add Channel • F-TMP-004 Translate • F-TMP-005 Editor • F-TMP-006 Formatting • F-TMP-007 AI – Draft First Version • F-SRC-001 Content Check • F-SRC-002 Filtering • F-SRC-003 Full Text Search • F-SRC-004 AI – Smart Ranking • F-AUT-005 Analysis • F-AUT-007 Reusable Component Creation • F-AUT-008 Citations & References • F-AUT-011 History Stretch — Nice to Have (3): • F-REV-011 AI – Comment to Action List • F-TMP-012 Conditional Render • F-COL-001 Peer Review

continued on next page

continued from previous page

Sprint	Focus	Deliverables
Sprint IV	<i>Channels, Audience & Template Library</i>	Committed (21 features): • F-CHN-001 Email • F-CHN-002 SMS • F-CHN-003 Moodle • F-CHN-004 Canvas • F-CHN-005 Calendar • F-CHN-006 Push Notifications • F-CHN-007 WhatsApp • F-CHN-008 Social Media • F-CHN-009 RSS / Atom Feed • F-CHN-010 Website • F-AUD-001 View • F-AUD-002 Copy • F-AUD-003 React / Like / Share / Comment • F-AUD-004 Bookmark • F-AUD-005 Progress • F-AUD-006 AI – Summarize • F-TMP-008 Library (Reuse Components) • F-TMP-009 Hyperlink & Asset Management • F-TMP-010 Tag & Cluster • F-TMP-011 AI – Convert Between Channel Templates • F-ADM-003 Monitor Stretch — Nice to Have (3): • F-AUD-012 Info Graph • F-AUD-013 Related Content • F-COL-002 Shared Space
Sprint V	<i>Advanced AI, Review Automation & Admin</i>	Committed (14 features): • F-AUD-007 AI – Clustering • F-AUD-008 AI – RAG • F-AUD-009 Note Taking • F-AUD-010 Content Filtering • F-AUD-011 AI – AI Tutor • F-REV-006 Assignment of Rules • F-REV-007 Comparison Between Versions • F-REV-008 Plagiarism • F-REV-009 AI – Screening • F-ADM-006 AI – Monitor • F-ADM-007 Setup Pipelines • F-ADM-008 Invalidation • F-ADM-009 AI – Analyse • F-AUT-009 AI – Update Old Templates / Drafts • F-AUT-010 AI – Content Pipeline Stretch — Nice to Have (2): • F-AUD-014 AI – Convert Content Template • F-COL-003 Q&A

7 Team

Name	Roll Number	Role
Sathwik Kodamarthi	cs23btech11025	Backend
Praneeth Chamarthi	cs23btech11012	Database, DevOps
Gona Sanjana	cs23btech11019	Backend
Jami Rithvik	cs23btech11022	Database, DevOps
Junjure Sathwik Reddy	es23btech11020	Frontend
Anushka Agrawal	es23btech11006	Frontend

Appendix A Glossary

Term	Definition
AI Quota	Per-user limit on AI requests, enforced at the Gateway to prevent overuse.
Application Layer	Manages high-level workflows and passes business logic down to the Service Layer.
Async AI Worker	A background process that runs long AI tasks without slowing down the main application.
Audit Log	A write-once record of system events, used for compliance and review.
Authentication	Verifying a user's identity at the Gateway by checking their token before anything else happens.
Authorisation (Object-Level)	A check in the Service Layer confirming whether a user is allowed to access a specific item.
Cache (Redis)	A fast in-memory store used to quickly serve frequently read data.
Canvas / Moodle	LMS platforms that receive published content via the Channels distribution module.
Channel	A named distribution endpoint (Email, SMS, etc.) with its own rendering configuration.
Clustering (AI)	AI-based grouping of content by topic, helping users discover related material.
Co-authoring (CRDT / OT)	Real-time editing by multiple users at once, with automatic conflict resolution.
Content Lifecycle States	Four governed states: Draft, In Review, Published, Archived. Transitions are validated.
CRUD	The four fundamental data operations: Create, Read, Update, Delete.
Data Layer	The storage tier: relational database, Redis cache, Search Index, and Vector Store.
DDoS	A large-scale attack that floods a service with traffic. Blocked at the Edge Layer.
Edge Layer	Outermost tier handling DDoS mitigation and bot filtering before the Gateway.

continued on next page

continued from previous page

Term	Definition
Event Bus	A message broker that passes events between services asynchronously, decoupling them from each other.
Functional Requirement	A specification of what the system must do, as a discrete capability.
Gateway Layer	The internal entry point for TLS, user authentication, API routing, and usage limits.
i18n (Internationalisation)	Organising UI text as key-value pairs that can be swapped per language, with a fallback for missing translations.
Intelligence Layer	A dedicated layer that runs heavy AI tasks, kept separate so they don't slow down normal API calls.
Outbox Pattern	Saves a data change and its notification event together in one step. A relay then sends it to the Event Bus.
Pipeline	A sequence of content-processing steps set up by an admin, with time limits and escalation rules per step.
RAG (Retrieval-Augmented Generation)	An AI technique that answers questions using passages retrieved from the library, reducing made-up answers and providing citations.
Role-Based Access Control	A system where permissions are attached to roles (Admin, Author, etc.) and users get those permissions through group membership.
Search Index	An index that powers full-text search with word stemming, synonyms, and typo correction.
SRS	A document that defines what a system must do and how well it must do it, written for all stakeholders.
Vector Store	A database of AI-generated content embeddings used for semantic search and question-answering (RAG).